Day 62 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 9, Day 7 reference

Watch: Types of RNN | Recurrent Neural Network Types | Deep Learning Tutorial 34 (codebasics) | Part Of Speech POS Tagging: NLP Tutorial For Beginners - S1 E11 (codebasics)

`day62_many_to_many_tagger.py`, segment by segment, in the order `main()` uses things. The code boxes are cut from the real script by program (inline comments removed so lines fit). Where a chunk implements a formula, a purple **MATH** box shows the formula first. Green **SYSTEM DIAGRAM** boxes show how the pieces fit, and the graphs are the ones the script itself saved. A glossary of new or reinforced syntax is at the end.

MATH BOX LEGEND

Symbols used in the MATH boxes: x_t = the vector for word t · h_t = hidden state after word t · W, b = weights and biases · p_t = predicted tag probabilities for word t · N = number of real (non-padding) words · d(something) = gradient of the loss with respect to it

Segment 0 — Imports and global setup

Imports and constants

CODE

```
import copy
import os
from collections import Counter, defaultdict
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
np.random.seed(42)
DATA_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ud_english_ewt_pos")
PAD, UNK = 0, 1
```

- `import copy` — gives **copy.deepcopy**, used to keep an independent snapshot of the best network.
- `from collections import Counter, defaultdict` — **Counter** counts items; **defaultdict** is a dictionary that creates a starting value the first time a missing key is used.
- `matplotlib.use("Agg")` — draw to image files with no window; must come before importing pyplot.
- `from matplotlib.patches import FancyBboxPatch` — a rounded rectangle shape, used to draw the parts diagram.
- `os.path.join(os.path.dirname(os.path.abspath(__file__)), ...)` — the dataset folder next to this script, wherever the script is run from.
- `PAD, UNK = 0, 1` — two constants set in one line: word id 0 is padding, word id 1 is “unknown word”.

Segment 1 — Part 1, from Day 56: helpers and the no-memory network

softmax and one_hot (unchanged from day56_exercise.py)

MATH

$$softmax(z)_k = exp(z_k) / \Sigma_j\ exp(z_j) \quad \textit{turns a row of scores into probabilities that sum to 1}$$

CODE

```
def softmax(z):
    z_shifted = z - z.max(axis=1, keepdims=True)
    exp_z = np.exp(z_shifted)
    return exp_z / exp_z.sum(axis=1, keepdims=True)
def one_hot(y, n_classes):
    out = np.zeros((y.shape[0], n_classes))
    out[np.arange(y.shape[0]), y] = 1
    return out
```

- `z.max(axis=1, keepdims=True)` — the largest value in each row, kept as a column so it subtracts row by row.
- `np.exp(z_shifted)` — e to the power of every entry. Subtracting the row maximum first keeps the numbers from overflowing.
- `exp_z / exp_z.sum(axis=1, keepdims=True)` — divide each row by its own total, so each row sums to 1.
- `np.zeros((y.shape[0], n_classes))` — one row per label, one column per class, all zero.
- `out[np.arange(y.shape[0]), y] = 1` — pairs of (row number, label): one cell per row is set to 1.

PlainFCNet (unchanged from day56_exercise.py)

MATH

$z_1 = X\,W_1 + b_1, \quad a_1 = max(0, z_1), \quad p = softmax(a_1\,W_2 + b_2)$

With a one-hot X, the product $X\,W_1$ simply picks one row of W_1.

CODE

```
class PlainFCNet:
    def __init__(self, T, hidden=16, n_classes=2, lr=0.2, seed=42):
        rng = np.random.RandomState(seed)
        self.W1 = rng.randn(T, hidden) * np.sqrt(2.0 / T)
        self.b1 = np.zeros(hidden)
        self.W2 = rng.randn(hidden, n_classes) * np.sqrt(2.0 / hidden)
        self.b2 = np.zeros(n_classes)
        self.lr = lr
    def forward(self, X):
        self.X = X
        self.Z1 = X @ self.W1 + self.b1
        self.A1 = relu(self.Z1)
        self.Z2 = self.A1 @ self.W2 + self.b2
        self.probs = softmax(self.Z2)
        return self.probs
    def backward(self, y_onehot):
        m = y_onehot.shape[0]
        dZ2 = self.probs - y_onehot
        dW2 = self.A1.T @ dZ2 / m
        db2 = dZ2.sum(axis=0) / m
        dA1 = dZ2 @ self.W2.T
        dZ1 = dA1 * drelu(self.Z1)
        dW1 = self.X.T @ dZ1 / m
        db1 = dZ1.sum(axis=0) / m
        self.W1 -= self.lr * dW1; self.b1 -= self.lr * db1
        self.W2 -= self.lr * dW2; self.b2 -= self.lr * db2
```

- `class PlainFCNet:` — copied without a single change. It is used today as the baseline with **no memory**.
- `rng.randn(T, hidden) * np.sqrt(2.0 / T)` — random starting weights, scaled by the number of inputs. Today **T** is the vocabulary size, because the input is one one-hot word.
- `X @ self.W1 + self.b1` — matrix product, then the bias added to every row (broadcasting).
- `self.probs - y_onehot` — the gradient of softmax plus cross-entropy with respect to the scores.
- `self.A1.T @ dZ2 / m` — weight gradient, averaged over the batch.
- `dA1 * drelu(self.Z1)` — pass the gradient back through ReLU: kept where the input was positive, zero elsewhere.
- `a -= b; c -= d` — two update statements on one line, separated by a semicolon.

Segment 2 — Part 2, from Day 57: the RNN forward pass

rnn_step_forward and rnn_forward

MATH

$$h_t = tanh(x_t W_{xh} + h_{t-1} W_{hh} + b_h)$$ *the same three weights at every word*

CODE

```
def rnn_step_forward(x_t, h_prev, Wxh, Whh, bh):
    return np.tanh(x_t @ Wxh + h_prev @ Whh + bh)
def rnn_forward(X, h0, Wxh, Whh, bh):
    m, T, _ = X.shape
    H = np.zeros((m, T, Whh.shape[0]))
    h = h0
    for t in range(T):
        h = rnn_step_forward(X[:, t, :], h, Wxh, Whh, bh)
        H[:, t, :] = h
    return H
```

- `np.tanh(x_t @ Wxh + h_prev @ Whh + bh)` — the whole recurrence in one line; unchanged from Day 57.
- `m, T, _ = X.shape` — CHANGED: X now has three axes (batch, time, embedding). _ is a name for a value not needed.
- `np.zeros((m, T, Whh.shape[0]))` — room for one hidden state per sentence per word.
- `X[:, t, :]` — CHANGED: all sentences' vectors for word t, shape (m, D). Day 57 used X[:, t:t+1] because each input was one number.
- `H[:, t, :] = h` — store the state; today every one of them is used to make a prediction.

Segment 3 — Part 3, from Day 58: backpropagation through time

rnn_backward (BPTT)

MATH

$$dh_t = dH_t + da_{t+1} W_{hh}^T \quad \textit{own output's gradient + gradient from the future}$$

$$da_t = dh_t \cdot (1 - h_t^2)$$

$$dW_{xh} \mathrel{+}= x_t^T da_t, \quad dW_{hh} \mathrel{+}= h_{t-1}^T da_t, \quad db_h \mathrel{+}= \Sigma\, da_t, \quad dx_t = da_t W_{xh}^T$$

CODE

```
def rnn_backward(dH, X, H, h0, Wxh, Whh):
    m, T, _ = X.shape
    dWxh = np.zeros_like(Wxh)
    dWhh = np.zeros_like(Whh)
    dbh = np.zeros(Whh.shape[0])
    dX = np.zeros_like(X)
    dh = np.zeros_like(h0)
    for t in reversed(range(T)):
        dh = dh + dH[:, t, :]
        h_prev = H[:, t - 1, :] if t > 0 else h0
        da = dh * (1.0 - H[:, t, :] ** 2)
        dWxh += X[:, t, :].T @ da
        dWhh += h_prev.T @ da
        dbh += da.sum(axis=0)
        dX[:, t, :] = da @ Wxh.T
        dh = da @ Whh.T
    return dWxh, dWhh, dbh, dX
```

- `np.zeros_like(Wxh)` — zeros with the same shape as Wxh: a running total for its gradient.
- `dh = np.zeros_like(h0)` — CHANGED: on Day 58 this started as the gradient at the last state. Now it starts at zero.
- `reversed(range(T))` — T-1, T-2, ..., 0: walk backward through the sentence.
- `dh = dh + dH[:, t, :]` — **NEW TODAY**: this word's own output adds its gradient. Written with **=** and **+**, not **+=**, so no array belonging to the caller is modified.
- `H[:, t - 1, :] if t > 0 else h0` — a one-line if/else: the previous state, or the initial state for the first word.
- `dh * (1.0 - H[:, t, :] ** 2)` — the derivative of tanh, computed from its stored output.
- `dWxh += X[:, t, :].T @ da` — add this word's contribution; the same weights were used at every word.
- `da.sum(axis=0)` — sum over the batch: one number per hidden unit.
- `dX[:, t, :] = da @ Wxh.T` — gradient for this word's vector, used later for the embedding table.
- `dh = da @ Whh.T` — hand the gradient to the previous word's state.

Segment 4 — Part 4, from Day 59: the gradient probe

grad_norm_by_timestep

CODE

```
def grad_norm_by_timestep(dH, H, Whh):
    T = H.shape[1]
    norms = np.zeros(T)
    dh = np.zeros_like(H[:, 0, :])
    for t in reversed(range(T)):
        dh = dh + dH[:, t, :]
        norms[t] = np.linalg.norm(dh)
        dh = (dh * (1.0 - H[:, t, :] ** 2)) @ Whh.T
    return norms
```

- `H.shape[1]` — the size of the time axis.
- `np.zeros_like(H[:, 0, :])` — a zero array shaped like one timestep's hidden states, (m, hidden).
- `np.linalg.norm(dh)` — the length of the whole array treated as one long vector (square, sum, square root).
- `(dh * (1.0 - H[:, t, :] ** 2)) @ Whh.T` — one step back in time: undo tanh, then through the recurrent weights.
- `return norms` — norms[0] is how much gradient reaches the first word.

Segment 5 — Part 5, from Day 60: gradient clipping

clip_by_global_norm (unchanged from day60_exercise.py)

MATH

$$||g|| = \text{sqrt}(\ \Sigma \text{ of every gradient entry squared }) \qquad \text{if } ||g|| > c: \quad g \leftarrow g \cdot c \,/\, ||g||$$

CODE

```
def clip_by_global_norm(grads, max_norm):
    norm = np.sqrt(sum(np.sum(g ** 2) for g in grads.values()))
    if norm <= max_norm:
        return grads, norm
    scale = max_norm / norm
    return {k: g * scale for k, g in grads.items()}, norm
```

- `grads.values()` — every gradient array in the dictionary.
- `sum(np.sum(g ** 2) for g in grads.values())` — a generator inside **sum**: add up the squares of every entry of every gradient.
- `return grads, norm` — two values returned as a tuple; below the threshold the original dictionary comes back untouched.
- `{k: g * scale for k, g in grads.items()}` — a dictionary comprehension: same keys, every array multiplied by the same factor.

Segment 6 — Part 6, from Day 61: words to padded arrays

read_tagged

CODE

```
def read_tagged(name):
    sentences, current = [], []
    with open(os.path.join(DATA_DIR, name + ".txt"), encoding="utf-8") as f:
        for line in f:
            line = line.rstrip("\n")
            if not line:
                sentences.append(current); current = []
            else:
                word, tag = line.split("\t")
                current.append((word.lower(), tag))
    return sentences
```

- `sentences, current = [], []` — two empty lists: finished sentences, and the one being read.
- `with open(..., encoding="utf-8") as f:` — open the file; it is closed automatically at the end of the block.
- `line.rstrip("\n")` — remove the newline at the end of the line.
- `if not line:` — an empty string counts as False, so this is true on a blank line: the sentence has ended.
- `sentences.append(current); current = []` — store the finished sentence and start a new empty one.
- `word, tag = line.split("\t")` — cut the line at the tab into exactly two pieces.
- `current.append((word.lower(), tag))` — add a (word, tag) pair, a tuple, with the word lowercased.

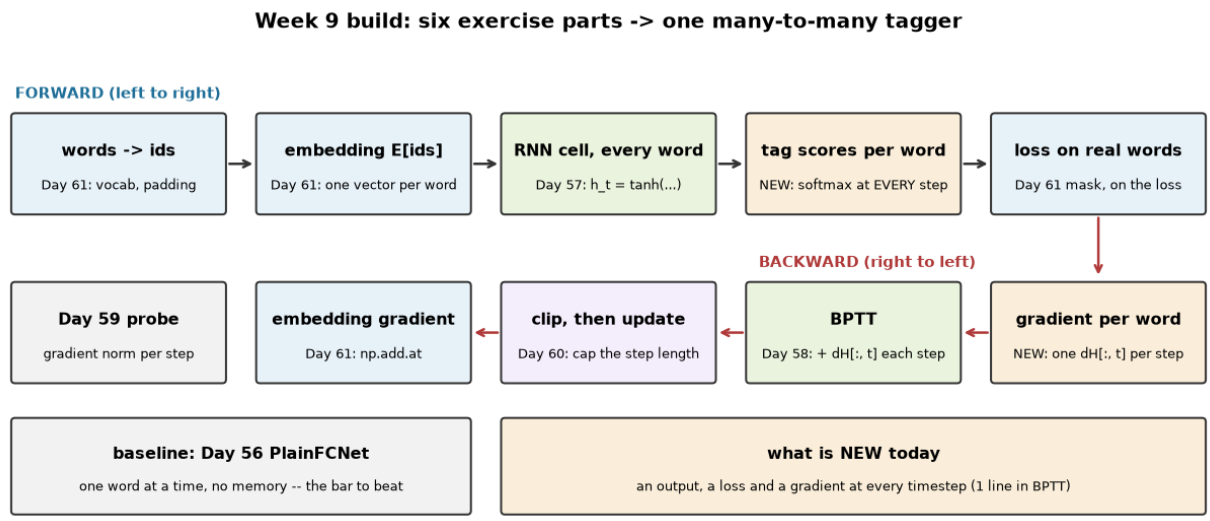
encode

CODE

```
def encode(sentences, stoi, ttoi):
    T = max(len(s) for s in sentences)
    ids = np.full((len(sentences), T), PAD, dtype=int)
    tags = np.zeros((len(sentences), T), dtype=int)
    lens = np.zeros(len(sentences), dtype=int)
    for row, sent in enumerate(sentences):
        ids[row, :len(sent)] = [stoi.get(w, UNK) for w, _ in sent]
        tags[row, :len(sent)] = [ttoi[t] for _, t in sent]
        lens[row] = len(sent)
    mask = np.arange(T)[None, :] < lens[:, None]
    return ids, tags, mask
```

- `max(len(s) for s in sentences)` — the length of the longest sentence.
- `np.full((n, T), PAD, dtype=int)` — an n-by-T array of integers filled with the padding id.
- `[stoi.get(w, UNK) for w, _ in sent]` — a list comprehension over (word, tag) pairs; **.get(w, UNK)** returns 1 for a word that is not in the vocabulary.
- `ids[row, :len(sent)] = [...]` — write the ids into the start of the row; the rest stays as padding.
- `[ttoi[t] for _, t in sent]` — the same for the tags; here the word is the part not needed.
- `np.arange(T)[None, :] < lens[:, None]` — positions as a row, lengths as a column; broadcasting compares every position with every length and gives an (n, T) True/False mask.

SYSTEM DIAGRAM



Segments 1–6 are the boxes of this diagram. The next segment wires them together.

Segment 7 — The assembled model: RNNTagger

RNNTagger.__init__

CODE

```
class RNNTagger:
    def __init__(self, vocab_size, n_tags, embed_dim=32, hidden_dim=32, lr=1.0, clip=1.0, seed=0,
        memory=True):
        rng = np.random.RandomState(seed)
        self.n_tags, self.hidden_dim = n_tags, hidden_dim
        self.lr, self.clip, self.memory = lr, clip, memory
        self.E = rng.randn(vocab_size, embed_dim) * 0.1
        self.Wxh = rng.randn(embed_dim, hidden_dim) * np.sqrt(1.0 / embed_dim)
        self.Whh = rng.randn(hidden_dim, hidden_dim) * np.sqrt(1.0 / hidden_dim) * float(memory)
        self.bh = np.zeros(hidden_dim)
        self.Why = rng.randn(hidden_dim, n_tags) * np.sqrt(1.0 / hidden_dim)
        self.by = np.zeros(n_tags)
```

- `memory=True` — a keyword argument with a default. Passing False switches the recurrent link off.
- `self.a, self.b = x, y` — store two settings on the object in one line.
- `rng.randn(vocab_size, embed_dim) * 0.1` — the embedding table: small random numbers, one row per word.
- `np.sqrt(1.0 / embed_dim)` — a starting scale that keeps signals a steady size through a tanh layer.
- `* float(memory)` — **float(True)** is 1.0 and **float(False)** is 0.0, so with memory off Whh starts at exactly zero.

RNNTagger.forward and loss

MATH

$$p_t = softmax(\, h_t W_{hy} + b_y \,) \quad for\ every\ word\ t$$

$$L = -(1 / N) \Sigma_{real\ words} \log p_t[\, true\ tag_t \,]$$

CODE

```
def forward(self, ids):
    m, T = ids.shape
    self.ids = ids
    self.h0 = np.zeros((m, self.hidden_dim))
    self.X = self.E[ids]
    self.H = rnn_forward(self.X, self.h0, self.Wxh, self.Whh, self.bh)
    self.H2 = self.H.reshape(m * T, self.hidden_dim)
    self.probs = softmax(self.H2 @ self.Why + self.by)
    return self.probs
def loss(self, tags, mask):
    p_true = self.probs[np.arange(tags.size), tags.ravel()]
    return -np.sum(np.log(p_true + 1e-9) * mask.ravel()) / mask.sum()
```

- `self.E[ids]` — fancy indexing: an (m, T) array of ids picks rows of E, giving (m, T, D).
- `rnn_forward(self.X, self.h0, ...)` — Part 2: every hidden state, shape (m, T, hidden).
- `self.H.reshape(m * T, self.hidden_dim)` — **NEW**: flatten batch and time into one axis, so each word is a row.
- `softmax(self.H2 @ self.Why + self.by)` — **NEW**: tag probabilities for every word at once, (m*T, n_tags).
- `tags.ravel()` — the (m, T) tag array flattened to m*T, in the same order as the rows of probs.
- `self.probs[np.arange(tags.size), tags.ravel()]` — for every row, the probability given to that word's true tag.
- `np.log(p_true + 1e-9) * mask.ravel()` — multiplying by the mask (True = 1, False = 0) removes padding from the sum.
- `/ mask.sum()` — divide by the number of real words: a mean over words, not over sentences.

RNNTagger.gradients

MATH

$$dlogits_t = (\, p_t - onehot(tag_t) \,) \cdot mask_t / N$$

$$dW_{hy} = \Sigma_t\, h_t^T\, dlogits_t, \quad dH_t = dlogits_t\, W_{hy}^T$$

CODE

```
def gradients(self, tags, mask):
    m, T = self.ids.shape
    dlogits = (self.probs - one_hot(tags.ravel(), self.n_tags)) * mask.ravel()[:, None] /
        mask.sum()
    grads = dict(Why=self.H2.T @ dlogits, by=dlogits.sum(axis=0))
    dH = (dlogits @ self.Why.T).reshape(m, T, self.hidden_dim)
    grads["Wxh"], grads["Whh"], grads["bh"], dX = rnn_backward(dH, self.X, self.H, self.h0,
        self.Wxh, self.Whh)
    grads["E"] = np.zeros_like(self.E)
    np.add.at(grads["E"], self.ids, dX)
    if not self.memory:
        grads["Whh"] = np.zeros_like(self.Whh)
    return grads
```

- `one_hot(tags.ravel(), self.n_tags)` — Part 1's helper, on the flattened tags.

- `mask.ravel()[:, None]` — the mask as a column, so it multiplies every column of a row.
- `dict(Why=..., by=...)` — build a dictionary using keyword arguments as keys.
- `(dlogits @ self.Why.T).reshape(m, T, self.hidden_dim)` — **NEW**: the gradient for each word's hidden state, put back into (batch, time, hidden) shape for BPTT.
- `a, b, c, dX = rnn_backward(...)` — Part 3 returns four things; three go straight into the dictionary.
- `np.zeros_like(self.E)` — an empty gradient table the same shape as the embedding.
- `np.add.at(grads["E"], self.ids, dX)` — add each word's gradient into its row; a word that appears twice gets both (Day 61).
- `if not self.memory:` — with memory off, replace Whh's gradient by zeros so Whh stays at zero.

RNNTagger.train_step and predict

CODE

```
def train_step(self, ids, tags, mask):
    self.forward(ids)
    loss = self.loss(tags, mask)
    grads = self.gradients(tags, mask)
    if self.clip is None:
        norm = np.sqrt(sum(np.sum(g ** 2) for g in grads.values()))
    else:
        grads, norm = clip_by_global_norm(grads, self.clip)
    for name, g in grads.items():
        getattr(self, name)[...] -= self.lr * g
    return loss, norm, self.clip is not None and norm > self.clip
def predict(self, ids):
    return self.forward(ids).argmax(axis=1).reshape(ids.shape)
```

- `if self.clip is None:` — **is None** tests for “no value”; here it means clipping is switched off.
- `grads, norm = clip_by_global_norm(grads, self.clip)` — Part 5; unpack the two returned values.
- `for name, g in grads.items():` — loop over (parameter name, gradient) pairs.
- `getattr(self, name)[...] -= self.lr * g` — fetch the weight array by its name and update it in place; **[...]** means all entries.
- `self.clip is not None and norm > self.clip` — True only if clipping is on **and** this step was long enough to be clipped.
- `.argmax(axis=1).reshape(ids.shape)` — the most probable tag for every row, folded back to (m, T).

Segment 8 — Training

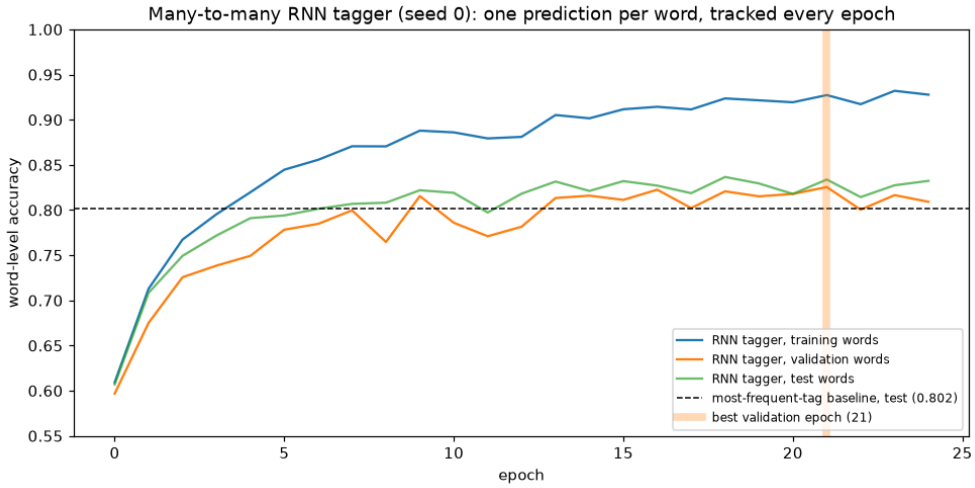
token_accuracy, fit and fit_cached

CODE

```
def token_accuracy(pred, tags, mask):
    return (pred == tags)[mask].mean()
def fit(data, vocab_size, n_tags, seed, epochs=25, batch_size=32, **net_kwargs):
    (ids_tr, tags_tr, mask_tr), (ids_va, tags_va, mask_va), (ids_te, tags_te, mask_te) = data
    net = RNNTagger(vocab_size, n_tags, seed=seed, **net_kwargs)
    rng = np.random.RandomState(seed)
    lens_tr = mask_tr.sum(axis=1)
    hist = dict(loss=[], train=[], val=[], test=[], clipped=[], max_norm=[])
    best_val, best_net, best_epoch = -1.0, None, -1
    for epoch in range(epochs):
        perm = rng.permutation(len(ids_tr))
        losses, norms, clips = [], [], []
        for start in range(0, len(perm), batch_size):
            idx = perm[start:start + batch_size]
            T_batch = lens_tr[idx].max()
            loss, norm, clipped = net.train_step(ids_tr[idx, :T_batch], tags_tr[idx, :T_batch],
                mask_tr[idx, :T_batch])
            losses.append(loss); norms.append(norm); clips.append(clipped)
        hist["loss"].append(np.mean(losses)); hist["clipped"].append(np.mean(clips));
            hist["max_norm"].append(np.max(norms))
        hist["train"].append(token_accuracy(net.predict(ids_tr), tags_tr, mask_tr))
        hist["val"].append(token_accuracy(net.predict(ids_va), tags_va, mask_va))
        hist["test"].append(token_accuracy(net.predict(ids_te), tags_te, mask_te))
        if hist["val"][-1] > best_val:
            best_val, best_net, best_epoch = hist["val"][-1], copy.deepcopy(net), epoch
    return best_net, best_epoch, {k: np.array(v) for k, v in hist.items()}
RUNS = {}
def fit_cached(data, vocab_size, n_tags, seed, name="left-to-right", **net_kwargs):
    key = (name, seed, tuple(sorted(net_kwargs.items())))
    if key not in RUNS:
        RUNS[key] = fit(data, vocab_size, n_tags, seed, **net_kwargs)
    return RUNS[key]
```

- `(pred == tags)[mask].mean()` — True/False per position, keep only the real words, take the mean: accuracy.
- `**net_kwargs` — collect any extra keyword arguments into a dictionary and pass them on to RNNTagger.
- `(a, b, c), (d, e, f), (g, h, i) = data` — nested unpacking of three (ids, tags, mask) tuples.
- `mask_tr.sum(axis=1)` — the number of True values in each row: each sentence's real length.
- `dict(loss=[], train=[], ...)` — a dictionary of empty lists, one per thing recorded each epoch.
- `rng.permutation(len(ids_tr))` — the sentence numbers in a fresh random order.
- `ids_tr[idx, :T_batch]` — pick this batch's rows and only the first T_batch columns: less padding, less work.
- `np.mean(clips)` — the mean of True/False values is the fraction that are True.
- `copy.deepcopy(net)` — a full independent copy of the network at its best validation epoch.
- `{k: np.array(v) for k, v in hist.items()}` — turn every recorded list into an array.
- `key = (name, seed, tuple(sorted(net_kwargs.items())))` — a tuple that identifies a run; tuples can be dictionary keys, dictionaries cannot.
- `if key not in RUNS:` — train only the first time these exact settings are asked for.

REAL GRAPH



Real graph from the script: what fit() recorded for seed 0, one point per epoch.

Segment 9 — Loading the data

prepare_data

CODE

```
def prepare_data():
    splits = {name: read_tagged(name) for name in ["train", "val", "test"]}
    counts = Counter(w for s in splits["train"] for w, _ in s)
    itos = ["<pad>", "<unk>"] + sorted(w for w, c in counts.items() if c >= 2)
    stoi = {w: i for i, w in enumerate(itos)}
    tag_names = sorted({t for s in splits["train"] for _, t in s})
    ttoi = {t: i for i, t in enumerate(tag_names)}
    data = [encode(splits[name], stoi, ttoi) for name in ["train", "val", "test"]]
    return splits, counts, itos, stoi, tag_names, ttoi, data
```

- `{name: read_tagged(name) for name in [...]}` — a dictionary comprehension: split name to its list of sentences.
- `Counter(w for s in splits["train"] for w, _ in s)` — two loops in one generator: every sentence, every (word, tag) pair, count the words.
- `sorted(w for w, c in counts.items() if c >= 2)` — words seen at least twice, in alphabetical order.
- `sorted({t for s in ... for _, t in s})` — **{ }** with a loop builds a **set**, which keeps each tag once; sorted turns it into an ordered list.
- `{t: i for i, t in enumerate(tag_names)}` — tag name to tag id.

Segment 10 — demo_gradient_check

numeric_gradient and the pass rule

MATH

$$\text{numeric slope} \approx (L(w + \varepsilon) - L(w - \varepsilon)) / 2\varepsilon$$

$$\text{pass if} \quad | \text{numeric} - \text{analytic} | \le 1e\text{-}10 + 1e\text{-}6 \cdot | \text{numeric} | \quad \text{for every entry}$$

CODE

```
def numeric_gradient(net, name, ids, tags, mask, eps):
    P = getattr(net, name)
    numeric = np.zeros_like(P)
    for pos in np.ndindex(P.shape):
        old = P[pos]
        P[pos] = old + eps; net.forward(ids); loss_up = net.loss(tags, mask)
        P[pos] = old - eps; net.forward(ids); loss_down = net.loss(tags, mask)
        P[pos] = old
        numeric[pos] = (loss_up - loss_down) / (2 * eps)
    return numeric
# inside demo_gradient_check():
diff = np.abs(numeric - analytic[name])
rel = diff / (np.abs(numeric) + np.abs(analytic[name]) + 1e-12)
ok = bool(np.all(diff <= 1e-10 + 1e-6 * np.abs(numeric)))
all_pass = all_pass and ok
if rel.max() > worst[0]:
    worst = (rel.max(), name, np.unravel_index(rel.argmax(), rel.shape))
```

- `getattr(net, name)` — the parameter array itself, so changing P changes the network.
- `np.ndindex(P.shape)` — every index of the array as a tuple, for any number of dimensions.
- `P[pos] = old + eps; net.forward(ids); loss_up = net.loss(tags, mask)` — three statements on one line: nudge, recompute, read the loss.
- `(loss_up - loss_down) / (2 * eps)` — the central difference: the measured slope.
- `np.all(diff <= 1e-10 + 1e-6 * np.abs(numeric))` — the pass rule, checked for every entry at once.
- `bool(...)` — turn NumPy's True/False into a plain Python one.
- `rel.argmax()` — the position of the largest value, counted as if the array were flattened.
- `np.unravel_index(i, shape)` — convert that flat position back to a (row, column) index.

Segment 11 — demo_baselines

Most-frequent-tag by counting

CODE

```
seen = defaultdict(Counter)
for w, t in zip(ids_tr[mask_tr], tags_tr[mask_tr]):
    seen[w][t] += 1
mft = np.full(V, seen[UNK].most_common(1)[0][0])
for w, c in seen.items():
    mft[w] = c.most_common(1)[0][0]
```

- `defaultdict(Counter)` — a dictionary whose missing keys start as an empty Counter.
- `zip(ids_tr[mask_tr], tags_tr[mask_tr])` — boolean indexing keeps the real words; zip pairs each word id with its tag id.
- `seen[w][t] += 1` — count this (word, tag) pair; works even the first time w is seen.
- `seen[UNK].most_common(1)[0][0]` — most_common(1) is a list with one (tag, count) pair; [0][0] takes the tag.

- `np.full(V, ...)` — an array of length V filled with that default tag.
- `mft[w] = c.most_common(1)[0][0]` — overwrite the default for every word that was seen.

Day 56's network, one word at a time

CODE

```
def one_hot_words(word_ids):
    return one_hot(word_ids, V)
def fc_predict(net, ids):
    flat = ids.ravel()
    out = [net.forward(one_hot_words(flat[i:i + 2000])).argmax(axis=1) for i in range(0,
        len(flat), 2000)]
    return np.concatenate(out).reshape(ids.shape)
fc = PlainFCNet(V, hidden=32, n_classes=K, lr=1.0, seed=7)
words, labels = ids_tr[mask_tr], tags_tr[mask_tr]
```

- `def one_hot_words(word_ids):` — a function defined inside another function; it can use V from the enclosing one.
- `ids.ravel()` — all positions as one long row.
- `range(0, len(flat), 2000)` — 0, 2000, 4000, ...: work in chunks so the one-hot matrix stays small in memory.
- `[... for i in range(...)]` — a list with one array of predictions per chunk.
- `np.concatenate(out).reshape(ids.shape)` — join the chunks and fold back to sentence shape.
- `mft[ids_te]` — (used just above in the script) indexing an array with an array of ids looks up every word's tag at once.
- `PlainFCNet(V, hidden=32, n_classes=K, lr=1.0, seed=7)` — the Day 56 class with different numbers; the code inside is untouched.

Segment 12 — demo_where_memory_helps

Word groups and the shuffle test

CODE

```
groups = {"all words": mask_te,
          "unambiguous": mask_te & (ids_te != UNK) & ~base["ambiguous"][ids_te],
          "ambiguous": mask_te & base["ambiguous"][ids_te],
          "unknown": mask_te & (ids_te == UNK)}
order = np.tile(np.arange(ids_te.shape[1]), (len(ids_te), 1))
for row, n in enumerate(mask_te.sum(axis=1)):
    order[row, :n] = rng.permutation(n)
ids_sh = np.take_along_axis(ids_te, order, axis=1)
tags_sh = np.take_along_axis(tags_te, order, axis=1)
```

- `mask_te & (ids_te != UNK) & ~base["ambiguous"][ids_te]` — **&** is element-wise AND, **~** is NOT. Brackets around each comparison are required.
- `base["ambiguous"][ids_te]` — a True/False array of length V, indexed by the word ids: is each test word ambiguous?
- `np.tile(np.arange(T), (n, 1))` — repeat the row 0..T-1 once per sentence: every row starts in its normal order.
- `enumerate(mask_te.sum(axis=1))` — row number together with that sentence's real length.
- `order[row, :n] = rng.permutation(n)` — shuffle only the real positions; padding positions keep their place.
- `np.take_along_axis(ids_te, order, axis=1)` — reorder each row by its own list of positions. Applying the same order to the tags keeps every word with its true tag.

Unknown words by the word before them

CODE

```
prev = np.full_like(ids_te, -1)
prev[:, 1:] = ids_te[:, :-1]
before_ids = [stoi[b] if isinstance(b, str) else b for b in before]
sel = groups["unknown"] & np.isin(prev, before_ids)
true_tag, true_n = Counter(tags_te[sel]).most_common(1)[0]
rnn_tag, rnn_n = Counter(p_rnn0[sel]).most_common(1)[0]
rnn_acc = np.mean([(p == tags_te)[sel].mean() for p in preds["RNN tagger"]])
```

- `np.full_like(ids_te, -1)` — an array shaped like ids_te, filled with -1 (“no word before”).
- `prev[:, 1:] = ids_te[:, :-1]` — columns 1 onward receive columns 0 to second-last: every position now holds the id of the word before it.
- `isinstance(b, str)` — is b a text string? Words are looked up in stoi; numbers (-1 and UNK) are used as they are.
- `np.isin(prev, before_ids)` — True wherever the previous word is one of the listed ids.
- `Counter(tags_te[sel]).most_common(1)[0]` — the commonest true tag in this group and its count, unpacked into two names.
- `np.mean([... for p in preds["RNN tagger"]])` — accuracy on this group, averaged over the three seeds.

Choosing the two example sentences by program

CODE

```
gain = ((p_rnn == tags_te) & (p_mft != tags_te) & mask_te).sum(axis=1)
loss_ = ((p_rnn != tags_te) & (p_mft == tags_te) & mask_te).sum(axis=1)
lens = mask_te.sum(axis=1)
short = np.where((lens >= 5) & (lens <= 9))[0]
pick_good = short[np.argmax(gain[short] - loss_[short])]
pick_bad = short[np.argmax(loss_[short] - gain[short])]
```

- `(p_rnn == tags_te) & (p_mft != tags_te) & mask_te` — real words the RNN gets right and the baseline gets wrong.
- `.sum(axis=1)` — count them per sentence.
- `loss_` — a trailing underscore avoids clashing with other things called loss.
- `np.where(cond)[0]` — the row numbers where the condition is True.
- `short[np.argmax(gain[short] - loss_[short])]` — among the short sentences, the one with the biggest net gain. Nothing was picked by hand.

Segment 13 — demo_parts_at_work

Running the probe on two losses

CODE

```
rows = np.where(mask_te.sum(axis=1) == 20)[0]
ids20, tags20 = ids_te[rows, :20], tags_te[rows, :20]
net.forward(ids20)
dlogits = (net.probs - one_hot(tags20.ravel(), K)) / tags20.size
dH_every = (dlogits @ net.Why.T).reshape(len(rows), 20, -1)
dH_last = np.zeros_like(dH_every); dH_last[:, -1, :] = dH_every[:, -1, :]
n_every = grad_norm_by_timestep(dH_every, net.H, net.Whh)
n_last = grad_norm_by_timestep(dH_last, net.H, net.Whh)
direct = np.array([np.linalg.norm(dH_every[:, t, :]) for t in range(20)])
```

- `mask_te.sum(axis=1) == 20` — sentences with exactly 20 real words, so no padding is involved.
- `ids_te[rows, :20]` — those rows, first 20 columns.
- `/ tags20.size` — .size is the total number of entries: the number of words here.
- `.reshape(len(rows), 20, -1)` — **-1** lets NumPy work out the last dimension (the hidden size).
- `dH_last[:, -1, :] = dH_every[:, -1, :]` — copy only the last word's gradient into an otherwise zero array.
- `np.array([np.linalg.norm(dH_every[:, t, :]) for t in range(20)])` — one norm per position, built with a list comprehension.

The clipping comparison

CODE

```
settings = [(32, 1.0), (32, 2.0), (64, 1.5), (64, 2.0)]
clip_res = {}
for hidden, lr in settings:
    for clip in [1.0, None]:
        runs = [fit_cached(data, V, K, seed=s, lr=lr, clip=clip, hidden_dim=hidden) for s in
            SEEDS]
        accs = [h["test"][be] for _, be, h in runs]
        clip_res[(hidden, lr, clip)] = accs
```

- `for hidden, lr in settings:` — each item is a pair, unpacked into two names.
- `[fit_cached(...) for s in SEEDS]` — three training runs; any that were already done are reused from the cache.
- `[h["test"][be] for _, be, h in runs]` — each run is (network, best epoch, record); take test accuracy at the best epoch.
- `clip_res[(hidden, lr, clip)] = accs` — a tuple used as a dictionary key.

REAL GRAPH



Real graph from the script. Left: Segment 12's groups. Middle: Segment 13's probe. Right: the clipping comparison.

Segment 14 — demo_limits

reverse_sentences and flipping predictions back

CODE

```
def reverse_sentences(ids, tags, mask):
    ids_r, tags_r = ids.copy(), tags.copy()
    for row, n in enumerate(mask.sum(axis=1)):
        ids_r[row, :n] = ids[row, :n][::-1]
        tags_r[row, :n] = tags[row, :n][::-1]
    return ids_r, tags_r, mask
data_rev = [reverse_sentences(*split) for split in data]
p_b = p_rev.copy()
for row, n in enumerate(n_real):
    p_b[row, :n] = p_rev[row, :n][::-1]
```

- `ids.copy(), tags.copy()` — work on copies so the original arrays are not changed.
- `ids[row, :n][::-1]` — **[::-1]** reverses: the first n entries of the row, last to first.
- `reverse_sentences(*split)` — ***** unpacks the (ids, tags, mask) tuple into three arguments.
- `p_rev[row, :n][::-1]` — reversing a second time puts the predictions back in normal word order.

The hardest tags

CODE

```
for k in range(K):
    sel = mask_te & (tags_te == k)
    stats.append(((p0 == tags_te)[sel].mean(), k, int(sel.sum())))
for acc, k, n in sorted(stats)[:6]:
    wrong = Counter(p0[mask_te & (tags_te == k) & (p0 != k)])
    w_tag, w_n = wrong.most_common(1)[0]
```

- `mask_te & (tags_te == k)` — real test words whose true tag is k.
- `stats.append((accuracy, k, count))` — a tuple per tag.
- `sorted(stats)[:6]` — tuples sort by their first item, so this is the six lowest accuracies.
- `Counter(p0[... & (p0 != k)])` — among the mistakes for tag k, count what was predicted instead.
- `w_tag, w_n = wrong.most_common(1)[0]` — the commonest wrong answer and how many times.

Segment 15 — The parts diagram

draw_parts_diagram

CODE

```
def draw_parts_diagram():
    fig, ax = plt.subplots(figsize=(13, 5.6))
    ax.set_xlim(0, 13); ax.set_ylim(0, 5.6); ax.axis("off")
    def box(x, y, w, h, title, sub, color):
        ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.04", facecolor=color,
            edgecolor="#333333", linewidth=1.2))
        ax.text(x + w / 2, y + h * 0.63, title, ha="center", va="center", fontsize=9.5,
            fontweight="bold")
        ax.text(x + w / 2, y + h * 0.27, sub, ha="center", va="center", fontsize=8)
    def arrow(x0, y0, x1, y1, color="#333333"):
        ax.annotate("", xy=(x1, y1), xytext=(x0, y0), arrowprops=dict(arrowstyle="->",
            color=color, linewidth=1.5))
```

- `ax.set_xlim(0, 13); ax.set_ylim(0, 5.6); ax.axis("off")` — a plain 13-by-5.6 drawing area with no axes shown.
- `def box(...):` — a helper defined inside the function, so it can use **ax** directly.
- `FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.04", ...)` — a rounded rectangle with its corner at (x, y).
- `ax.add_patch(...)` — put the shape on the drawing.
- `ax.text(x + w / 2, y + h * 0.63, title, ha="center", va="center")` — text centred on a point inside the box.
- `ax.annotate("", xy=..., xytext=..., arrowprops=dict(arrowstyle="->"))` — an arrow with no text, from xytext to xy.

Quick Syntax Glossary — new or reinforced constructs

- `dh = dh + dH[:, t, :]` — the many-to-many line of BPTT: each timestep adds its own gradient.
- `H.reshape(m * T, hidden)` — treat every word of every sentence as its own row, so 2-D helpers work unchanged.
- `probs[np.arange(n), labels]` — pick one entry per row: the probability of each row's true class.
- `x * mask / mask.sum()` — zero out padding and average over real items only.
- `np.arange(T)[None, :] < lens[:, None]` — a padding mask built by broadcasting, no loop.
- `defaultdict(Counter)` — nested counting without checking whether a key exists yet.
- `counter.most_common(1)[0][0]` — the single commonest item in a Counter.
- `array[other_array_of_ids]` — look up many values at once (mft[ids_te], base["ambiguous"][ids_te]).
- `a & b, ~a` — element-wise AND and NOT on True/False arrays; wrap each comparison in brackets.
- `np.take_along_axis(a, order, axis=1)` — reorder each row by its own index list.
- `np.tile(row, (n, 1))` — repeat a row n times to make a 2-D array.
- `np.isin(a, values)` — True where an entry of a is one of the listed values.
- `row[:n][::-1]` — reverse just the first n entries.
- `f(*tuple_of_args)` — unpack a tuple into separate arguments.
- `x is None / x is not None` — the correct test for “no value”.
- `FancyBboxPatch, ax.annotate` — rounded boxes and arrows for a diagram drawn with matplotlib.

End of Day 62 syntax reference. For the experiments and what they showed, see the Day 62 complete guide.